Clase 1

@August 31, 2026

Introducción

El objetivo principal de este curso es crear desde cero un motor gráfico 3D casero usando C++ y DirectX. Durante este cuatrimestre y el siguiente, trabajaremos en los cimientos del motor abarcando temas clave como: optimización, renderizado de polígonos, manipulación de gráficas 3D, UI/UX, shaders, y físicas.



Desarrollo

Configuración del Entorno y Verificación

Visual Studio Morado

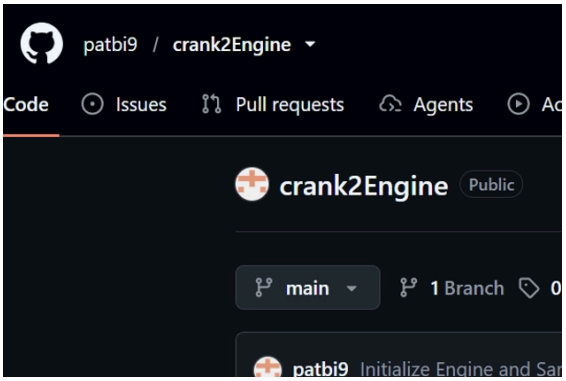
Para desarrollar el motor, hay que instalar las herramientas de **Desktop development with C++** desde el Visual Studio Installer. Utilizaremos la versión 2026.

Hardware

Ejecutamos el comando `dxdiag` para verificar las capacidades de nuestro equipo.

Mi compu THUNDER tiene un procesador Intel Core i9, 32GB de RAM, y una **NVIDIA GeForce RTX 4070**, lo cual es excelente para desarrollar gráficos con DirectX 12/11.

GITHUB



El primer paso administrativo fue crear un repositorio en GitHub para llevar el control del código del motor.

> *Nota: El motor del profe Charreton se llama NovaEngine*

Arquitectura del Proyecto: Motor vs Sandbox

Para este curso, trabajaremos creando una **Solución en Visual Studio que contendrá 2 proyectos distintos.**

En el desarrollo profesional de software, un motor gráfico nunca está mezclado con el código del juego. Se separan para que el motor pueda ser reutilizado en múltiples proyectos. Por lo tanto, nuestra arquitectura se divide en:

El Motor Gráfico (Engine):

Es el corazón del proyecto. Aquí se encapsula la API gráfica (**DirectX 11**), el sistema de renderizado, el ECS (Entity Component System), matemáticas, audio y gestión de recursos.

No es un ejecutable (EXE). Se compilará como una **DLL** (Dynamic-Link Library).

El Sandbox / Editor (crank2Engine):

Esta es la aplicación que sí se ejecuta (el EXE). Es la "caja de arena" o zona de pruebas.

Contiene el `WinMain` (el punto de entrada en aplicaciones Windows), se encarga de crear la ventana del sistema operativo y **"consume" o importa las herramientas del Motor Gráfico** para renderizar y probar las cosas en pantalla.

Conclusión

En esta clase hicimos las configuraciones y las primeras bases para el motor. Y definimos como se va a separar la solución de visual studio.

Glosario

Rendering Engine Development Job:

Trabajo especializado de los videojuegos o VFX. Enfoque en diseñar, programar, y optimizar el código encargado de comunicarse con la tarjeta de video para renderizar los gráficos 3D y 2D en pantalla de la manera más eficiente posible, calculando luces, sombras, y geometrías en tiempo real.

dxdiag (DirectX Diagnostic Tool)

Herramienta en Windows que diagnostica y muestra información sobre el hardware del sistema y los componentes multimedia. Sirve para confirmar que el equipo soporta las versiones necesarias de DirectX.

API (Application Programming Interface)

Funciones e instrucciones que Microsoft proporciona para que el código C++ pueda "hablar" directamente con la tarjeta GPU sin tener que programar en el lenguaje nativo del hardware.

Solución (en Visual Studio)

Es un contenedor que agrupa uno o más "Proyectos" que están relacionados entre sí para construir una aplicación completa.

DLL (Dynamic-Link Library)

Es un archivo que contiene código, datos y funciones compiladas que pueden ser utilizadas por otros programas. En lugar de copiar todo el código del motor dentro de tu juego, el juego simplemente enlaza y llama a las funciones de la DLL mientras se está ejecutando.

.LIB (Library)

En el contexto de vincular una DLL a tu proyecto, el archivo `.lib` actúa como un índice o un "mapa". Le dice a tu aplicación ejecutable (Sandbox) dónde encontrar y cómo conectarse a las funciones que están guardadas dentro del archivo `.dll` al momento de compilar.